

UQMES QA walkthrough

A primer for inspectors and managers

System	UQMES — Demo Company tenant
Walks against	WO-QA-INSPECT-01 • Common Rail Injector • 8 parts
Audience	QA Inspectors and QA Managers
Contents	Sections 1–16, plus sidebar appendix

Contents

The shape of a QA day	1
Getting around — the sidebar	1
1. Your home page — orient yourself	4
2. Receiving inspection — a lot of injectors arrives	6
3. First Piece Inspection buy-off — INJ-QA-INSPECT-001	8
4. A sampled part comes to you — INJ-QA-INSPECT-002	10
5. A part fails your inspection — INJ-QA-INSPECT-003	12
6. Working the disposition	14
7. Re-inspecting a reworked part — INJ-QA-INSPECT-004	17
8. OSP return inspection — INJ-QA-INSPECT-005	19
9. Working a CAPA task — CAPA-2024-002 and CAPA-2024-004	21
10. Calibration awareness	26
11. The notification bell and inbox	29
12. Reading the audit trail	31
13. The manager's side — Maria	33
14. Authoring a DWI — the process flow	36
15. Quality Reports	40
16. Glossary	42
Appendix — Sidebar reference	43

Who this is for. The QA team getting oriented in UQMES — inspectors and managers both. It follows one inspector (Sarah) hands-on through a demo work order, because a single narrative is the clearest way to learn the system; but the decisions she hands up — the first-piece buy-off, the disposition, CAPA verification and approval — are the QA **Manager's** authority, and the walk names who owns each gate. Read as an inspector, it's your day end to end. Read as a manager, watch the gates — they're where your sign-off sits. Read it once start-to-finish; refer back by section later.

The shape of a QA day

Two roles, one flow. Work moves through the shop and QA gates it at the points that matter.

- **As an inspector**, your day is a queue: check your inbox → buy off first pieces → inspect sampled and failed parts → disposition the failures → re-inspect reworked parts → open or work CAPAs when a pattern emerges. \$1–12 walk exactly that, in order.
- **As a manager**, your day is mostly the *other side of those gates* — the approvals and verifications inspectors send up. You authorize disposition decisions, verify whether a CAPA actually worked, approve major CAPAs before work can start, and co-sign at an inspector's station when they lack the authority themselves. Those hinge points are flagged where they occur — \$3 (buy-off), \$6 (disposition), \$9 (CAPA) — and **\$13 walks them end to end from the manager's chair**.

Getting around — the sidebar

You land on the home page (\$1); the collapsible **sidebar rail** on the left is how you reach everything else. Two things to know before the walk sends you into it:

- **It's permission-gated.** You only see the sections your role can use, so an inspector's rail is shorter than a manager's, and a manager's is shorter than a tenant admin's. If this guide names a section you don't see, your role doesn't have it — that's expected, not a bug.
- **Two sections start expanded** — Production and Quality, a QA person's home turf. The rest (Supply, Approvals, Remanufacturing, Admin) start collapsed; click the section header to open one.

That's all you need to start. The full rail — every section with its key routes — is in the **Sidebar reference** appendix at the end; refer back to it whenever a surface sends you somewhere unexpected.

What this is not. A training curriculum for a trainer to teach with (see [QA_INSPECTOR_TRAINING_SCRIPT.md](#) for that — it carries the role-play, checkpoints, and gotcha essays). This is a self-serve reference: shorter, first-person, with fewer exits into pedagogy.

What you'll be walking. A dedicated demo work order, [WO-QA-INSPECT-01](#) (Midwest Fleet Services · Common Rail Injector · 8 parts), is seeded into the Demo Company tenant specifically for this walk. Each

part is pre-staged into the state a section walks against, so the exhibits are always there; the demo tenant is reset to this state before each run.

Scannable traveler PDF for the walk: [artifacts/WO-QA-INSPECT-01_traveler.pdf](#). Print it (or open it on a phone) if you want to physically scan the header barcode / QR to open the live WO — the scan resolves to the same WO Detail page you’d reach by clicking. It also includes the full 12-operation routing table with sign-off blocks, a paper counterpart to what’s on screen.

Roles you’ll play. Passwords are [demo123](#).

Email	Name	Role	Where you play it
sarah.qa@demo.ambac.com	Sarah Chen	QA Inspector	Every section — the walker’s identity.
maria.qa@demo.ambac.com	Maria Santos	QA Manager	§13 (the manager’s whole side), and the gates in §3/§6/§9 when Sarah hands one up.
mike.ops@demo.ambac.com	Mike Rodriguez	Operator	§3 — the seed pre-signs the first-piece substeps as Mike so Sarah (playing QA) can sign off the FPI without hitting the segregation-of-duties gate. You don’t log in as Mike; his signatures are already on the seed exhibit.

Sarah’s QA Inspector role has [sign_off_fpi](#) (§3) and [close_disposition](#) (§6d), and covers almost the whole walk on her own. The one gate she can’t clear solo is the **disposition decision** (§6b): choosing a disposition type — *any* type, REWORK included — is gated by [approve_disposition](#), which is SOD-restricted to QA Manager / Tenant Admin. She isn’t blocked, though: the editor opens an inline **co-sign** dialog and an authorized colleague (Maria) approves it right there, recorded under their name (§6b, §13b). So keep Maria’s login handy for §6; you don’t otherwise switch users. (If your tenant’s role config narrows other actions to QA Manager too, log in as Maria — the walk still works.)

The parts on WO-QA-INSPECT-01, and where each is used:

Part	Where it sits in the seed	Section
INJ-QA-INSPECT-001	Nozzle Inspection · PENDING FPI · first piece designated	3
INJ-QA-INSPECT-002	Nozzle Inspection · AWAITING_QA · sampled (“Post-repair verification”)	4
INJ-QA-INSPECT-003	Flow Testing · IN_PROGRESS · fresh, ready for a live FAIL	5

Part	Where it sits in the seed	Section
INJ-QA-INSPECT-004	Flow Testing · AWAITING_QA · visit 2, historical FAIL QR + CLOSED REWORK disposition already on file	7
INJ-QA-INSPECT-005	Nitride Coating · RETURNED from Apex Plating · awaiting return inspection	8
INJ-QA-INSPECT-006	Assembly · QUARANTINED · bare OPEN NCR assigned to Sarah	§1 (background)
INJ-QA-INSPECT-007 , INJ-QA-INSPECT-008	Cleaning / Disassembly · IN_PROGRESS	filler, not walked

1. Your home page — orient yourself

Log in as sarah.qa@demo.ambac.com. You land on / — Sarah’s QA home, which also answers at </quality/inbox> (the name \$11 uses). The *Approvals* and *CAPA tasks* tiles below link to a separate generic </inbox>.

Top-left header. [Welcome back, Sarah](#) and an [Incoming queue](#) button that jumps to </production/incoming>.

Scan box. A single input labeled “Scan or type a work order / part number...” with a [Go](#) button (disabled until you type or scan something). Scans always resolve to the parent work order and drop you on WO Detail ([/workorder/\\$id](/workorder/$id)) — the shared work surface where you can pick up any part on that WO. Part scans go to the part’s parent WO, not to the part detail.

FPI banner (red border, on-screen heading *First piece waiting*). Every pending First Piece Inspection in your tenant surfaces here as a row: step name, work order, part (if designated), and how long it’s been waiting. Two buttons per row:

- **I’m on it** — acknowledges the pending FPI so the operator sees QA is on the way. After you click it, the row reads “*Seen by Sarah*”.
- **Start check** — opens the work order’s Control page, which has a pending first-piece panel you can buy off from directly. (\$3 also walks the fuller “sign at the operator’s station” path.)

For this walk the FPI banner shows two rows: the pre-existing [WO-2024-0048-A](#) row (from a different demo storyline) and your [WO-QA-INSPECT-01](#) · [INJ-QA-INSPECT-001](#) row. The 001 row is \$3.

Inbox with chips. A flat list of everything QA owes a decision on, grouped by four filter chips: *All* · *Receiving* · *OSP returns* · *In-process*. Each chip carries its row count and the age of its oldest item ([Receiving 5](#) · [3d](#)), with a tooltip spelling that out (“*oldest: 3d*”). Rows are clickable — clicking navigates you to the appropriate work surface for that row’s type.

“Urgent” is a badge, not a chip. A red [Urgent 1](#) pill sits after the four chips, counting anything past its own age threshold. It is plain text, not a button — you cannot filter by it, so don’t go hunting for the click target.

For this walk the Inbox shows (among others):

- Receiving lots from Great Lakes Diesel and Bargain Bolts (\$2).
- OSP-return shipments from Apex Plating. Yours for \$8 shows as [OSP-2026-000003](#) — the UI displays the sequential shipment number, not the seeder’s [reference](#) (see 8a).
- An in-process row for [WO-QA-INSPECT-01](#) · [Nozzle Inspection](#) · [1 pcs](#) — that’s [INJ-QA-INSPECT-002](#), the sampled part for \$4.

My Quality Actions. Three tiles counting your assigned items: *Approvals* (approvals waiting for your signature), *CAPA tasks* (Containment / Corrective / Preventive actions assigned to you), and *My dispositions* (Quarantine dispositions assigned to you). Where each one actually takes you is worth knowing, because two of the three share a destination:

- *Approvals* → [/inbox](#)
- *CAPA tasks* → [/inbox](#)
- *My dispositions* → [/production/dispositions](#), **unfiltered** — the tile counts your open ones, but the page it opens lists every quarantined part in the tenant (see 6a).

The *My dispositions* tile currently reads **2** — it shows only the open (not-yet-closed) dispositions assigned to you. The two you see on a fresh seed:

- **DISP-QAI-006-OPEN** — OPEN, no type yet, on **INJ-QA-INSPECT-006** (the seed's background exhibit; the walk doesn't drive it).
- an IN_PROGRESS SCRAP disposition on an older-storyline part — auto-created from a FAIL QR, so its number is auto-assigned (**DISP-2026-0000NN**) and shifts on each reseed; don't key on the exact number.

The **DISP-QAI-004-REW** rework disposition is CLOSED and does NOT contribute to this count, even though it's assigned to Sarah — that's by design; a closed disposition isn't work waiting.

Your gauges. Calibration status on gauges you've used recently. Currently reads "*Torque Wrench TW-25 — overdue 15d*". Real day: a gauge overdue for calibration should not be used until re-calibrated; a link to [/quality/calibrations](#) sits here to check status.

You will return to this home page repeatedly through the walk. It's your dashboard. The home page is where you *land*; the left **sidebar rail** is how you reach every other surface this walk names — mapped in full in the **Sidebar reference** appendix.

2. Receiving inspection — a lot of injectors arrives

Real day terms: a pallet of Common Rail Injectors from Great Lakes Diesel has arrived at the receiving dock. You need to sample and inspect it against the sampling plan before it becomes available inventory.

2a — Reach the receiving queue

Either click the **Receiving** chip on your home Inbox (it reads [Receiving 5 · 3d](#) — 5 rows, oldest 3 days), or navigate to </production/receiving-inspection> directly from the URL bar.

Two queues, and they are not the same page. Worth getting straight before you start, because the home page and this section send you to different ones:

- </production/receiving-inspection> — the *Receiving Inspection Queue*. **Purchased lots only**; 5 rows on a fresh seed.
- </production/incoming> — *Incoming Inspection*, the unified queue: purchased lots **and** parts back from a subcontract vendor (the same 5 lots plus the OSP returns awaiting inspection, so a few rows more), with a *Source* column and *All sources / All statuses* filters. This is where the home page's **Incoming queue** button goes, and where \$8 picks up the OSP return.

Both open the same inspection runtime; the unified one is just a wider net. This section uses the receiving-only queue.

You land on: </production/receiving-inspection> — a table with columns *Lot # · Material · Supplier · Qty · Status · Actions*. Every row has an **Inspect** button, and the header carries **Import / Export** plus a *Search receiving inspection queue...* box.

Rows include several [RCV-INJ-000#](#) lots from Great Lakes Diesel in [AWAITING_INSPECTION](#), and one [RCV-INJ-HOLD](#) from Bargain Bolts in [QUARANTINE](#) with reason “*Unqualified supplier*” — that row exists to illustrate the supplier-hold state; don't inspect it.

2b — Open a lot

Click **Inspect** on [RCV-INJ-0001](#) (Great Lakes Diesel, 250 EA).

You land on: [/production/receiving-inspection/\\$lotId](/production/receiving-inspection/$lotId) — the lot detail page.

You see:

- Header: [RCV-INJ-0001](#) · status [AWAITING_INSPECTION](#) · **Documents** button (attach cert of conformance, packing slips, etc.).
- Subheader: [Common Rail Injector](#) · [Great Lakes Diesel](#) · [qty 250](#).
- “**No Certificate of Conformance captured for this lot.**” with an **Upload CoC** button. In a real receiving workflow the CoC is uploaded here first; the demo lets you proceed either way.
- **Sample plan (C0, level III, TIGHTENED)** panel: [Inspect 29 of 250](#) · [Accept \{}gl{\le} 0](#) · [Reject \{}gl{\ge} 1](#). Zero-acceptance sampling at inspection level III, tightened switching — a single defect rejects the whole lot.

- A note: “*This receiving step has digital work instructions. Run the inspection through the operator runtime.*”
- **Run Inspection (DWI)** button.

2c — Run the DWI

Click **Run Inspection (DWI)**.

You land on: the operator substep runtime, scoped to this lot.

You see the DWI-guided *Inspect incoming material* form. Fields:

- **Scan the lot / packing slip** — barcode / QR input, optional.
- **Outer Diameter** (RCV-01) — required, spec 25 +0.05 / \{}gl{-}0.05 mm.
- **Incoming inspection result** — required, buttons Pass / Fail / Pending.
- **Inspection sign-off** — required, “Sign as detected by” button.
- **Defects found** — “No defects found.” with “Add defect” button. Only used on FAIL.

Top of the runtime carries stepper navigation — **Jump to substep 1: <name>** for each substep plus **Jump to review**. This lot’s DWI has one substep, so you’ll see one of each.

The footer does not block you. It names what’s outstanding — “3 required fields missing — Outer Diameter, Incoming inspection result, ...” — but **Confirm & review stays enabled**. Clicking it with fields missing scrolls you to the first one rather than submitting (its tooltip reads “Tap to scroll to: ...”). Only **Back** is disabled, because this is the first substep. So an unfilled form doesn’t produce a dead button; it produces a jump.

2d — Pass the lot

Enter a passing value and sign:

- **Outer Diameter:** 25.01 (well within spec).
- **Incoming inspection result:** click **Pass**.
- **Sign as detected by:** click to sign.

Click **Confirm & review**. On the review pane, click **Accept lot** (receiving-specific verb; the general operator runtime uses “Complete step”, but receiving shows **Accept lot** / **Reject lot** to match the domain).

Toast: “Lot accepted” — you land back on the lot detail page, which now reflects the completed inspection.

The lot leaves **AWAITING_INSPECTION** and becomes stock available for a work order.

Fail flow: click **Fail** instead of Pass, add a defect (Type + Description), then complete. That opens the Reject disposition dialog for type + severity + quantity. This walk doesn’t drive that path here — Sarah’s in-process fail path is \$5.

3. First Piece Inspection buy-off — INJ-QA-INSPECT-001

Real day terms: on a WO's first pass through a step, one part is designated as the **first piece**. Operators run the DWI on that piece, then QA reviews the inspection result and signs off before the rest of the batch can be run through the step. The FPI is a production gate: a failed or missing FPI blocks the whole batch.

3a — Spot the FPI on your home

Your home page's red-bordered **First piece waiting** block shows your pending FPI row: [WO-QA-INSPECT-001](#) · [INJ-QA-INSPECT-001](#), Nozzle Inspection, ~6h waiting.

Click **I'm on it** first — this acknowledges the pending FPI, so the operator sees you're on the way. The row updates to read *"Seen by Sarah"*.

3b — Reach the runtime

Start check on your home's FPI banner takes you to the work order's Control page, which has a **pending first-piece** panel — the quick, QA-native way to buy off without opening the operator's runtime at all. This section instead walks the fuller path — signing at the operator's station, so you see what the operator sees:

1. Go to WO Detail ([/workorder/\\$id](/workorder/$id)) and click **Start Work** (top-right).
2. In the dialog, check the [INJ-QA-INSPECT-001](#) row under Nozzle Inspection and click **Start**.
3. The runtime opens. Every inspection substep already carries a completion **signed by Mike** — the seed pre-populates those so you (Sarah, playing QA) can go straight to the buy-off.

What the runtime looks like. The runtime hydrates Mike's prior first-piece captures, so the header reads *"2 of 2 confirmed"* and the measurement/inspection fields show his recorded values (e.g. Spray Angle 15°). You don't need to enter anything to buy off — the FPI banner is independent of the substep form.

If you paste a runtime URL directly, include [workOrder](#). The FPI banner shows only when the URL carries a [workOrder](#) param; reach the runtime any other way and you'll see the DWI with no banner and no explanation. Going through **Start Work** sets it for you.

Segregation-of-duties (SOD) note: the person who signed the first-piece substeps cannot also sign off the FPI. That's why the seed uses Mike (operator) for the substeps and expects Sarah (QA) for the buy-off. If you re-sign any substep yourself, the FPI Pass endpoint will return 400: *"Segregation of duties: this user signed one or more of the first piece's inspection substeps. FPI buy-off must be signed by a different qualified inspector."*

3c — Sign off the FPI

On the runtime, the FPI banner shows three action buttons: **Sign off & pass** · **Fail** · **Waive**.

Choose:

- **Sign off & pass** — records the FPI as PASSED. Opens a confirmation dialog with an optional notes field: *“By signing off you attest that the setup is correct and the first piece (INJ-QA-INSPECT-001) conforms. This is recorded against your name and releases the run.”* Click **Confirm sign-off**. The batch is released; other parts can now run through Nozzle Inspection.
- **Fail** — records FAILED. The batch is blocked pending investigation. A FAILED FPI usually indicates a setup problem and often triggers a CAPA.
- **Waive** — records WAIVED with a required reason (≥ 10 characters). Rare; a waived FPI still counts as a documented decision.

For this walk: click **Sign off & pass**, optionally add a note like *“Nozzle geometry matches drawing rev; spray-hole bank clear.”*, click **Confirm sign-off**. Toast: *“FPI signed off — parts can now proceed.”*

Recording the pass **is** the step’s QA sign-off for the first-piece run (recorded against your name), so the step is no longer blocked on it. The FPI banner clears from your home page and turns green on the runtime — *“First Piece Inspection signed off · Setup verified — all parts can proceed through this step.”*

Permission note. The Pass / Fail / Waive verdicts are gated server-side on the `sign_off_fpi` permission. Someone who holds it (Sarah in this seed) signs off directly. Someone who *doesn’t* isn’t a dead end: the banner offers a **second-person co-signature** — an authorized QA person authenticates inline at that same station (`cosign_email` / `cosign_password`), is never logged in, and the verdict is recorded against **them**. QA can also work their own queue without touching the operator’s session via the pending-FPI panel on WO Control. (§4 lands on that same Control page too — but for a sampled inspection, not an FPI buy-off.)

Manager’s side. Co-signing at Sarah’s station is the *inline* half of your authority; §13e contrasts it with the *async* half — requests that land in your own `/approvals` queue. §13 walks the manager’s whole day.

4. A sampled part comes to you — INJ-QA-INSPECT-002

Real day terms: sampling rules flag certain parts for QA inspection mid-process. This part is flagged with reason “*Post-repair verification*” — it had earlier rework at an earlier step, so the sampling rule marked it for extra scrutiny when it reached its next QA-gated step (Nozzle Inspection).

4a — Find it on your Inbox

On your home page, click the **In-process** chip (or leave it on All). The row “*Nozzle Inspection · 1 pcs · Common Rail Injector · WO-QA-INSPECT-01 · WO due 2026-08-10*” is INJ-QA-INSPECT-002.

Click the row.

You land on: [/workorder/\\$id/control](/workorder/$id/control) for WO-QA-INSPECT-01 — the Control page. In the Step Status table, INJ-QA-INSPECT-002 is at Nozzle Inspection · [AWAITING_QA](#) · flagged **Sample**. Its [sampling_context](#) reads [Post-repair verification](#).

4b — Open the part detail

From Control, the per-part table row’s serial is plain text — Control doesn’t link out to part detail. To reach detail, switch to WO Detail ([/workorder/\\$id](/workorder/$id)), open the Parts tab, and click the ExternalLink icon on the INJ-QA-INSPECT-002 row.

You land on: [/details/Parts/\\$id](/details/Parts/$id) — the part detail page.

You see:

- Header: [INJ-QA-INSPECT-002](#) · [Common Rail Injector](#).
- **General:** Status [AWAITING_QA](#).
- **Production:** Order (link to Orders detail), Current Step [Nozzle Inspection](#) (link to Steps detail), Work Order ([WO-QA-INSPECT-01](#), link — this jumps to Control, not Detail).
- **Quality Control:** [Sampling Required](#) · Yes, [Sampling Reason](#) · [post repair verification](#). [Rework Passes](#) · 1 reflects an earlier rework cycle at another step.

4c — Run the inspection

Order matters: \$3 signed off the Nozzle Inspection FPI. If you haven’t done \$3 yet, do it first — the FPI banner gates the whole step, and you’ll see “*First Piece Inspection in progress*” on part 002’s runtime too, blocking your sampled inspection.

Open the runtime from **WO Detail** ([/workorder/\\$id](/workorder/$id)): click **Start Work** in the header, tick this part, and **Start**. (The Control page’s Step Status rows carry step-routing controls — send to QA, reassign, previous step — *not* a runtime launcher, and the part-detail page doesn’t launch the runtime either; Start Work on WO Detail is the entry point.)

The DWI at Nozzle Inspection walks: visual inspection points on the 3D model (nozzle tip / spray-hole bank / seat face), a spray-angle measurement, and an inspector sign-off. Work through the captures and **Confirm & next** to the review pane, then click **Complete step**. Toast: *“Step complete — lot advanced (1 part moved).”*

The part advances to the next step in the process ([AWAITING_QA](#) → [IN_PROGRESS](#)).

5. A part fails your inspection — INJ-QA-INSPECT-003

Real day terms: you're inspecting a part at Flow Testing. You take the flow rate reading and it's out of spec. The system records the FAIL and immediately quarantines the part.

5a — Reach the runtime

From your Inbox, click any WO-QA-INSPECT-01 row (or use the scan box with [WO-QA-INSPECT-01](#)) to reach the WO Detail. In the header click **Start Work**, tick **INJ-QA-INSPECT-003** (grouped under Flow Testing), and **Start** — this creates the step-execution and opens its runtime. (Start Work lives on the WO Detail header, not the Parts tab.)

5b — Enter an out-of-spec value

The verdict here is **measurement-driven**: you record the flow reading, and the FAIL is *derived* from it being out of spec — there is no manual Pass/Fail button and no separate defect form on this substep. (The DWI vocabulary does support an explicit pass/fail node that writes straight to the report, but this step isn't authored with one.)

You see the *Flow test* substep with, top-to-bottom:

- A green **"First Piece Inspection signed off · Setup verified"** banner (seeded PASSED FPI on this step, so you're not blocked).
- **Rework attempt N of 2** counter — this WO's rework escalation threshold (N reflects the part's prior rework count).
- Decision point notice: *"Auto · QA result — routes automatically from the inspection result when you complete the step — pass takes Assembly, fail takes Rework."*
- **Scan the part barcode** — required (Barcode / QR).
- **Flow Rate (F-04)** — required, spec [120 +20 -20 mL/min](#) (i.e. LSL 100, USL 140).
- **Flow bench in-calibration** — required confirmation checkbox.

Enter values to trigger the FAIL:

- **Scan the part barcode**: any string like [INJ-QA-INSPECT-003](#).
- **Flow Rate**: **98**. Inline validation flags red — below LSL.
- **Flow bench in-calibration**: check the confirmation.

Click **Confirm & review** → **Complete step**. The out-of-spec flow rate resolves the report to FAIL automatically.

Toast: *"FAIL recorded — part held for disposition"* (red/error toast) with the description line listing the specific blockers — *"Part is quarantined and step blocks on quarantine; One or more measurements are out of specification; No active StepExecution"* (the exact blocker list varies with state). The system distinguishes hard-fail states from awaiting-signoff states in the toast heading, so you can tell at a glance a fail was recorded (not a benign timing wait).

5c — What this triggers

Filing the FAIL kicks off the nonconformance chain:

- A **Quality Report** (QR-2026-#####, FAIL) is created for INJ-QA-INSPECT-003 at Flow Testing.
- A **Quarantine Disposition** is auto-created (OPEN, no type yet) and assigned to a QA Manager / Inspector — the record \$6 works against.
- The part is **quarantined**, and a notification fires.

It surfaces immediately: your home *My dispositions* tile count goes up, and the part detail shows Latest Inspection **FAIL** · 1 open defect, Has Open Defect **Yes**, and the linked QR + disposition rows.

You just caused the disposition \$6 walks against.

6. Working the disposition

Real day terms: you filed a FAIL. Now you have to decide what to do with the part — rework, scrap, use as-is with concession, return to supplier, etc. That decision goes on the disposition record.

6a — Open the disposition

Two paths, either works:

- From **My dispositions** tile on your home → </production/dispositions>. Be ready for what this opens: the page is titled **Quarantined Parts** and it is a *parts* list, not a disposition list — ERP ID · Status · Step · Part Type · Process · Created At · Details · Actions, with *Archived / Requires Sampling / Needs QA / Exclude Terminal* filters. It is **not** filtered to you despite the tile's count, so search for [INJ-QA-INSPECT-003](#). Each row's action is either **Disposition** (no disposition yet — creates one) or **Edit Disposition** (one exists). You want Edit Disposition.
- From the part detail page → **Dispositions** widget → edit icon on the OPEN row. Fewer rows to wade through; prefer this one.

You land on: [/dispositions/edit/\\$id](/dispositions/edit/$id) — the disposition editor.

You see:

- Header with disposition number and part.
- **Disposition Details** form top-to-bottom:
 - **Current State** — dropdown, currently [OPEN](#). Setting a Disposition Type below auto-transitions this to [IN_PROGRESS](#) on save.
 - **Disposition Type** — dropdown, currently empty. See 6c for what each type means.
 - **Severity** — dropdown, default [MAJOR](#), with the label “*Severity classification of the nonconformance*” underneath.
 - **Assigned To** — pre-set to whoever was assigned at auto-create.
 - **Description** — auto-populated from the QR.
 - **Containment Action** — free text. **Required to close** for MAJOR/CRITICAL severity.
 - **Resolution Notes** — free text for what you decided and why.
 - **Requires Customer Approval** — checkbox; auto-set for USE_AS_IS and REPAIR, both of which need a customer concession.
 - **Related** — links to the linked Quality Report(s) and 3D Annotations widget.

6b — Fill in the decision

For this walk, pick **REWORK**:

- **Disposition Type:** [REWORK](#).
- **Severity:** [MAJOR](#).
- **Containment Action:** “*Part quarantined and segregated pending rework at Flow Testing.*”

- **Resolution Notes:** *“Retest after cleaning; suspect fouling in the seat.”*

Click **Update Disposition**. Signed in as Sarah — who lacks `approve_disposition` — the editor doesn’t commit yet: it opens the **Authorize disposition decision** co-sign dialog (*“Recording a ‘Rework’ disposition needs approval authority. An authorized colleague can co-sign here; the decision is recorded under their name.”*). Enter an authorized approver’s email (`maria.qa@demo.ambac.com`), draw the signature, tick *“I authorize this disposition decision,”* enter their password (`demo123`), and click **Authorize**. Toast: *“Disposition updated”* — the disposition moves to `IN_PROGRESS`, recorded under Maria. A QA Manager working it directly holds the authority and commits without the dialog (§13b).

Why the co-sign. Per AS9100/ISO 9001 8.7 the disposition decision carries a signature, and choosing a type — *any* type, not just `USE_AS_IS` — is that authorized act. An inspector without `approve_disposition` co-signs it to an authorized colleague inline rather than being blocked. `USE_AS_IS` and `REPAIR` additionally require a recorded customer/design-approval reference (they accept known-nonconforming product).

Once authorized, the disposition moves to `IN_PROGRESS`, and — because the part is still quarantined — `REWORK` sends it back for rework (status → `REWORK_NEEDED`, rework count +1). If the part had already moved on, the decision is recorded as paper only and the part stays put.

Manager’s side. If you hold `approve_disposition`, setting the type commits directly — no co-sign dialog, because the authority is already yours. §13b covers your side, including the customer-reference rule that `USE_AS_IS` and `REPAIR` enforce.

6c — The doors, briefly

- **REWORK** — send back through the rework loop. Most common. Sends the part to `REWORK_NEEDED` (paper-only if the part has already moved on) and increments the rework count.
- **REPAIR** — accept with repair outside normal spec; may not fully conform (AS9100). Same cascade behavior as `REWORK`.
- **USE_AS_IS** — accept the non-conformance under a customer concession. Requires a recorded customer/design-approval reference, **enforced** at decision time: the `decide` action refuses a `USE_AS_IS` (or `REPAIR`) with no reference. Not a shortcut.
- **SCRAP** — terminal. Part status cascades to `SCRAPPED` from any state (terminal-rank precedence still applies — a `SCRAPPED` part can’t be pulled back by a later `REWORK`).
- **RETURN_TO_SUPPLIER** — return under SCAR to the original supplier. Terminal for internal; part status cascades to `CANCELLED` from any state.

6d — Close the disposition

Once the rework has been done and re-inspected (§7 walks that), close the disposition — either from its **Close** action or by setting **Current State** → `CLOSED` in the editor and clicking Update. Both run the same checks: closing is gated by the `close_disposition` permission, and it’s refused unless the completion blockers are clear — containment recorded for MAJOR/CRITICAL, a disposition decision selected, and no pending 3D annotations. If a blocker is unmet (e.g. a MAJOR with no containment), the close is rejected

with the blocker and the disposition stays open; any other edits in that submit (containment text, notes) are still saved, so you fix the blocker and retry.

7. Re-inspecting a reworked part — INJ-QA-INSPECT-004

Real day terms: a part that previously failed has been reworked and is back at the same step for a second inspection (visit 2). The audit trail on the part detail shows the full arc: the original FAIL QR, the CLOSED REWORK disposition, and now this visit-2 inspection.

7a — Find it

How this connects to \$6d. You just closed the disposition on INJ-QA-INSPECT-003. \$7 walks a *different* part (INJ-QA-INSPECT-004) that's already been through that same close in the seed — its [DISP-QAI-004-REW](#) is pre-CLOSED and the rework has been done offline. This is what a real re-inspection looks like the day the reworked part comes back to your bench.

Two clean paths:

- From your home Inbox — the WO-QA-INSPECT-01 In-process rows include another one at Flow Testing. That's INJ-QA-INSPECT-004.
- Or navigate to [/workorder/\\$WO-QA-INSPECT-01/control](/workorder/$WO-QA-INSPECT-01/control) and find the 004 row in the Step Status table.

7b — Read the part's history first

Open the part detail (via the WO Detail Parts tab's ExternalLink icon). You see:

- **Status:** [AWAITING_QA](#) (Flow Testing).
- **Quality Reports:** 1 row — [QR-QA-INSPECT-004-FT](#) · [FAIL](#) · [Flow Testing](#), description “*Flow rate 98 mL/min - below LSL of 100 mL/min. Reworked and returned for re-inspection.*” — click to open the failing report.
- **Dispositions:** 1 row — [DISP-QAI-004-REW](#) · [CLOSED](#) · [REWORK](#) · [MAJOR](#) — the signed rework decision, with resolution notes describing the nozzle replacement. **This is the record to read.**
- **Rework Passes:** 1 — the rework counter incremented once, from the original REWORK cascade.

7c — Run the re-inspection

Open the runtime via **Start Work** on WO Detail (\$5a), picking INJ-QA-INSPECT-004. Enter a **passing** value — like the failing pass in 5b, the verdict is measurement-driven (no Pass/Fail button):

- **Scan the part barcode:** [INJ-QA-INSPECT-004](#).
- **Flow Rate:** 121 mL/min (in spec, LSL 100 / USL 140 → PASS derived).
- **Flow bench in-calibration:** check the confirmation.

Click **Confirm & review** → **Complete step**. Toast: “*Step complete — lot advanced (1 part moved).*”

What happens. A second QR ([QR-2026-...](#), **PASS**) is written and the part advances to the next step (Assembly). The rework arc is now paper-complete: FAIL QR → CLOSED REWORK → reworked → PASS QR.

The **Rework Passes** counter stays at 1 — it counts rework *cycles* (incremented when the REWORK disposition was applied in §6), not re-inspection passes.

8. OSP return inspection — INJ-QA-INSPECT-005

Real day terms: parts sent out to a subcontractor (Apex Plating, for Nitride Coating) have returned. Before accepting them back into the process, QA runs a receiving-style inspection on the outgoing/incoming characteristics — most importantly **Coating Thickness**.

8a — Find the returned shipment

Click the **OSP returns** chip on your home Inbox. The most-recent [Nitride Coating · OSP-2026-000003 · Apex Plating Co · returned](#) row is your shipment. (Shipment numbers auto-generate per tenant; the demo seeder tags this one with `reference=OSP-QA-INSPECT-01` internally, but the UI displays the sequential `OSP-2026-000003`. Older seed shipments — `OSP-2026-000001`, `-000002` — are separate storylines.)

Click the row.

You land on: [/production/incoming](#) (the incoming queue) with the shipment visible.

Alternatively, reach the shipment directly from the WO Detail: a small “**1 at outside process**” badge next to the WO header links to Control, where the Outside processing panel lists the shipment with an **Inspect** button.

8b — Open the return inspection

From either surface, click **Inspect** on the `OSP-2026-000003` row.

You land on: the operator substep runtime, scoped to the shipment.

You see the return-inspection DWI, one substep titled “*Return inspection (post-coating)*” with:

- **Coating Thickness** (`OSP-01`) — required, spec `12 +3 \{}gl{-}2 \{}gl{\mu}m`.
- **Photograph the coated surface** — optional image capture (tap to capture or upload).
- **Return inspection result** — required, Pass / Fail / Pending buttons.
- **Return inspection sign-off** — required, Signature field (“*Click to sign*”).

Enter a passing value:

- **Coating Thickness:** `12.5`.
- **Return inspection result:** **Pass**.
- **Return inspection sign-off:** click to sign.

Click **Confirm & review**. On the review pane the action buttons are **Accept return** / **Reject** (domain-specific — the OSP step uses Accept/Reject rather than the generic “Complete step” the operator runtime uses on non-OSP steps). Click **Accept return**.

Toast: “*Accepted — parts advanced past the outside-process step.*” You land on [/production/outside-processing](#) — the OSP board.

What happens. The part becomes **READY_FOR_NEXT_STEP** and stays at the Nitride Coating (OSP) step — it's *cleared to advance*, not moved to Final Test in the same click; a later advance carries it onward. The shipment transitions **RETURNED $\rightarrow$ CLOSED** — the OSP cycle is finished. (So the toast's "advanced past the outside-process step" is the intent, not an immediate step change.)

9. Working a CAPA task — CAPA-2024-002 and CAPA-2024-004

Real day terms: a disposition handles *this part right now*. A CAPA (Corrective And Preventive Action) handles *the pattern* — why is this happening again, what will we change so it stops. \$5 through 7 walked one failed part. This section walks the parallel system that catches the pattern behind repeated failures.

QA inspectors don't own CAPA *closure* ([verify_capa](#) is gated to the QA Manager), but they do the legwork: work assigned tasks, record verification data, and — when a QR reveals a systemic issue rather than a one-off — initiate a new CAPA.

Sarah has pre-seeded work across the five demo CAPAs. This section walks two of them in depth (CAPA-2024-004, then -002) and uses a third, CAPA-2024-003, to show how a multi-person task behaves.

9a — Find your CAPA work

On the home page, the **My quality actions** panel holds three counters: **Approvals**, **CAPA tasks**, and **My dispositions**. The **CAPA tasks** count covers every task Sarah owns — as the primary assignee, or as one assignee on a multi-person task. Expect five or six on a fresh seed; the exact number shifts because due dates are seeded relative to today, so don't treat it as a fixed expectation.

Sidebar → **Quality** → **CAPAs** (</quality/capas>) opens the full list: four stat cards (Active / Pending Verification / Overdue / Closed) above a table. Controls are a **Search capas...** box, a **Sort by...** dropdown, **New CAPAs**, a **Needs My Approval** toggle, and a **View CAPA** button on each row. There is no “assigned to me” filter, so read the *Assigned To* column.

The Status column is computed, not stored. It's derived from the CAPA's underlying facts — verification confirmed → Closed, all tasks done

- RCA complete → Pending Verification, any task or RCA started → In Progress, nothing yet → Open. So a CAPA that has tasks never displays as Open. What you'll actually see:
- **CAPA-2024-002** — Pending Verification, Preventive Action, Minor. Assigned to Sarah, all tasks complete. This is the verification exhibit.
- **CAPA-2024-003** — In Progress, Corrective Action, Major. Nozzle batch defects; Sarah owns two multi-assignee tasks.
- **CAPA-2024-004** — In Progress, Corrective Action, Major. Contamination investigation; Sarah has “Conduct contamination analysis”, Not Started.
- **CAPA-2024-005** — In Progress, Corrective Action, Major. Customer return, assigned to Sarah; no RCA yet.

9b — Complete an assigned task (CAPA-2024-004)

Two places can complete a task, and they now behave identically:

- the **Inbox**, which lists only *your* outstanding tasks — the fastest route when you're working your own queue;

- the CAPA detail **Tasks** tab, which lists *every* task on the CAPA grouped by type (Containment / Corrective / Preventive). Each row carries three controls, all icon-only and easy to miss: a **checkbox** on the left that opens the completion dialog, and a **pencil** (edit) and **trash** (delete) at the right. Use this when you're looking at the CAPA as a whole.

Either way, completion applies the same rules — the assignee mode and any signature requirement — from both entry points.

Walking it from the Inbox:

1. Sidebar → **Inbox** (or the **CAPA tasks** counter on the home page). Header reads “1 overdue, 8 total items”.
2. Tabs across the top: **All, Tasks, Approvals, Dispositions**. Cards are grouped by urgency — **Overdue, This Week, Upcoming** — and those headings are collapsible buttons, so not every card is on screen at once. If a task you expect is missing, expand the other groups before concluding it isn't there.
3. Find “Conduct contamination analysis” — badged *Corrective Action*, referencing CAPA-2024-004, assigned to Sarah Chen, Not Started, Due in 6 days. Each card offers **View** and **Complete**.
4. Click **Complete**. The **Complete Task** dialog opens — “Mark this task as complete and add any notes about what was done.” Fill **Completion Notes** (placeholder: “Describe what was done to complete this task...”). **Attach Evidence** is available if you have a measurement printout or photo.
5. Click **Complete Task**. The task is stamped with the completion date and your name, and the CAPA's progress percentage ticks up.

A third task you didn't expect. CAPA-2024-004's Tasks tab shows **three** rows, not the two the seed lists: **T001** is “Containment: Increased cleaning solution filtration and monitoring”, auto-created from the CAPA's immediate action. Every CAPA with an immediate action gets one.

Multi-person tasks (CAPA-2024-003). Open CAPA-2024-003's Tasks tab (the tab beside it is labelled **Root Cause**, not RCA). Two of Sarah's tasks are multi-person:

- “Update incoming inspection procedure” — needs **all** assignees (Sarah AND Maria both must sign off).
- “Implement tightened sampling for nozzles” — needs **any** assignee (Sarah OR Jennifer, whichever gets there first).

Completing an all-assignees task records *your* sign-off but leaves the task open until every assignee has done the same; an any-assignee task closes on the first. The Tasks tab shows the mode per row (*Single Owner* on the ordinary ones), and the completion dialog says so up front for an all-assignees task rather than letting you discover it afterwards.

So on T005 “Update incoming inspection procedure”: Sarah completing it leaves the row **Not Started** with her assignee signoff recorded and Maria's still outstanding. That's correct, not a failed click.

9c — Record verification data (CAPA-2024-002)

CAPA-2024-002 shows **Pending Verification** — all tasks done and RCA complete, so someone now has to check whether the correction actually worked. Progress reads 75%.

Verification is **two stages**: you write the *plan* (how you'll measure it) first, and the outcome is recorded against that plan afterwards. You can't record a result for a measurement you never defined — which is the point.

1. Open **CAPA-2024-002** and click the **Verification** tab. It's headed "*Effectiveness Verification — Verify that corrective/preventive actions have been effective*", and on a fresh seed reads "*No verifications have been recorded yet.*"
2. Click **Add Verification**. The **Add Verification Plan** dialog opens — "*Define how you will verify the effectiveness of corrective actions.*"
3. Fill both required fields:
 - **Verification Method** — "*How will effectiveness be verified? (e.g., process audit, data review, etc.)*" For this CAPA: "Monitor next 30 days of receiving for missing-document holds."
 - **Success Criteria** — "*What defines success? (e.g., zero defects over 30 days, process capability > 1.33)*" Here: "Zero missing-document holds over a 30-day window."
4. Click **Create**. (**Create** stays disabled until both fields have content.)

Once the monitoring window has elapsed, record the outcome from the same tab: the plan row shows a **Complete Verification** button that opens a dialog for `effectiveness_result` (CONFIRMED / NOT_EFFECTIVE) plus notes. CAPA-2024-001 is the worked example: open its Verification tab to see a completed plan with a CONFIRMED result.

Who can verify. Sarah can add and edit the verification *plan*, but recording the *outcome* requires verification authority (`verify_capa`, held by a QA Manager). She isn't blocked: it's co-signable — she completes it with a QA Manager authenticating inline, and it's recorded against them. A QA Manager working their own queue records it directly.

What verifying does. CONFIRMED closes the CAPA. NOT_EFFECTIVE reopens it to In Progress, flags the RCA for review, and auto-creates a 30-day follow-up task — a correction that didn't stick doesn't quietly close. The initiator/assignee can't verify their own CAPA unless self-verification is enabled (with justification).

Manager's side. Recording the outcome is yours (`verify_capa`); §13c walks it. Separately, a MAJOR/CRITICAL CAPA needs your *management approval* before work even starts — a distinct gate that lands in your `approvals` queue, walked in §13d.

9c-bis — The other four tabs

§9b and §9c cover Tasks and Verification. The rest, briefly, so nothing on the page is a surprise:

- **Root Cause** — on a CAPA with no RCA yet this reads "*No root cause analysis has been performed yet*" with a **Start RCA** button. Worth knowing that an RCA is not optional decoration: the computed status can't reach Pending Verification without one (see 9a), so a CAPA whose tasks are all done but which still shows In Progress is usually missing its RCA.
- **Approval** — for a MAJOR/CRITICAL CAPA this shows an "*Awaiting Approval — This CAPA is pending management approval. Work cannot begin until approved.*" banner plus an Approval History list. Read-

only for Sarah; approving is a QA Manager action, walked from the manager’s side in §13d. A MINOR CAPA shows *Not Required* instead.

- **Documents** — **Attach Document**: a file picker, a **Classification** dropdown (PUBLIC / INTERNAL / CONFIDENTIAL / RESTRICTED / SECRET, defaulting to INTERNAL) and **Upload**. This is where inspection evidence lives if you didn’t attach it from the completion dialog.
- **History** — “*Timeline of changes and updates to this CAPA.*” On the seeded CAPAs this reads “*No audit history.*” for the same reason the seeded parts do (see §12c): the seeder writes rows directly rather than going through the runtime.

9d — Initiate a CAPA from a failed QR

Sarah has `add_capa` (the basic create permission every staff role has) plus `initiate_capa` (the business-verb permission that layers on top). Together those let her create new CAPAs. Operators have `add_capa` but *not* `initiate_capa`, so they can help edit a CAPA draft someone else opened but can’t create a new one — formal CAPA initiation sits with QA staff and supervisors, matching the sibling `close_capa` / `approve_capa` / `verify_capa` gates.

Initiating is the right move when a QR reveals a systemic issue, not a one-off part defect. From `/quality/capas` click **New CAPA**. Required inputs: problem statement, `capa_type` (CORRECTIVE/PREVENTIVE), severity (MINOR/MAJOR/CRITICAL), initial `assigned_to`. Optional: linked quality reports (link the failing QR you’re reacting to), work order, step, part.

On save:

- If severity is MAJOR or CRITICAL, the CAPA needs management approval before work can begin.
- An initial CONTAINMENT task is auto-created.
- The assignee gets a notification.

When to open one. Don’t create a CAPA for every failed QR — the disposition already records what to do with *this part*. Open a CAPA when there’s a pattern: “we’ve seen this three times in a month,” “customer complaint traced to a systemic gap,” “supplier’s process changed and we missed it.” §5’s disposition on INJ-QA-INSPECT-003 was a one-off; CAPA-2024-003 was the right response to the *fifth* nozzle failure in an order.

9e — CAPAs you didn’t open (quality gates)

Not every CAPA in your queue was raised by a person. A step can carry a **quality gate**: when an aggregate metric crosses a configured threshold — say fail rate over a rolling window — the gate fires its configured actions. One of those raises a CAPA (a SCAR).

A gate-raised CAPA looks different from one you filed:

- **Initiated By is empty**, and notification templates render it as “*System*”. That’s deliberate — the gate raised it, no human did. The trip is automatic and doesn’t depend on anyone holding `initiate_capa`; an operator’s permissions can’t suppress a quality gate.

- **The problem statement names the gate and the numbers** — “Auto-raised by quality gate ‘RS-Nozzle’ at step Nozzle Inspection: *FAIL_RATE_PCT* = 25.000 crossed threshold 10.000.” With no initiator to ask, the record has to explain itself.
- **It’s assigned to a QA Manager** (falling back to a QA Inspector), so it lands in a real queue and fires `capa.assigned` rather than sitting unnoticed.
- **A supplier-type gate makes it a SCAR** against the lot’s supplier instead of an internal CORRECTIVE.

Reconstructing why it fired. The gate firing records the ruleset, metric, computed value, threshold, actions taken, and the QR that tripped it. That QR’s *detected-by* is whoever was working when the threshold crossed. So even with no initiator on the CAPA, the chain CAPA ← firing → report → inspector reconstructs the full story. Don’t read *detected-by* as “the person who caused this” — they filed one inspection; the gate fired on the aggregate.

10. Calibration awareness

Real day terms: every measurement is only as good as the gauge you took it with. If the flow bench you used yesterday was drifting out of tolerance, everything you signed against it is suspect. UQMES tracks calibration state and surfaces it in three places for QA inspectors.

The current enforcement scope. UQMES blocks measurements written against equipment whose status is `OUT_OF_SERVICE` — the picker hides those options, and the server refuses the write (with the equipment name and reason) even if a stale client somehow selects one. `OUT_OF_SERVICE` is set by a FAIL calibration.

What's NOT blocked: a measurement written against a gauge that's still `IN_SERVICE` but whose calibration is *due-soon* or *overdue*. That's a softer signal — the gauge-nag tile flags those on the home page as an awareness prompt, but the picker doesn't hide them. If you find out after the fact that a gauge you used was overdue, use the QR void flow to walk the reading back.

10a — Your gauge-nag tile on the home page

The home page has a **Your gauges** tile beside the **My quality actions** panel. It counts gauges Sarah used in the last 7 days whose calibration is due within 7 days or already overdue.

Empty state reads “*Nothing you've used in the last 7 days is due for calibration.*” Populated, it reads “*N gauges you used in the last 7 days need calibration within 7 days*” with the top 3 listed inline — overdue rows render red as “overdue Nd”, due-soon rows as “due in Nd”. On a fresh seed you'll see exactly one: “*Torque Wrench TW-25 — overdue 15d*”. The **Review calibrations** button navigates to </quality/calibrations>.

10b — The calibration dashboard

</quality/calibrations> is the full QA view.

Top row — five stat cards (fresh-seed values in brackets):

- **Equipment** — with calibration records [5].
- **Current** — in calibration [4].
- **Due Soon** — within 30 days [2].
- **Overdue** — past due [1].
- **Compliance** — percent in compliance [80%].

Below — three panels:

- **Quick Actions** — three links:
 - *View All Calibration Records* → </quality/calibrations/records>.
 - *Record New Calibration* → </CalibrationRecordForm/new>.
 - *Manage Equipment* → </editor/equipment>.

- **Due Soon** — top 5 records due within 30 days.
- **Overdue** — top 5 records past due, rendered with destructive styling and “N days overdue” counter.

Two reports available from the header: **Calibration Due** and **Checking Aids**.

The **records list** (</quality/calibrations/records>) is the browse view behind that first Quick Action. Columns: *Equipment, Result, Status, Calibration Date, Due Date, Type, Certificate #, Actions*. It has a **Search calibration records...** box, a **Sort by...** dropdown, **New Calibration Records**, and per-row **Edit / Delete**. On a fresh seed there are five rows — the same five gauges as the dashboard’s Equipment card, with *Torque Wrench TW-25* showing Result *Pass* but Status **Overdue**. That pairing is worth pausing on: the gauge passed its *last* calibration, it’s simply due for the next one. Result and Status answer different questions.

10c — Recording a new calibration

Record New Calibration opens the calibration-record form in create mode. Its fields:

- **equipment** (FK) — which piece of equipment this event is for.
- **calibration_date** / **due_date** — when it happened, when it’s due next.
- **result** — PASS / FAIL / LIMITED.
- **calibration_type** — SCHEDULED / INITIAL / AFTER_REPAIR / etc.
- **performed_by** — user reference.
- **external_lab** / **certificate_number** — if calibrated outside.
- **standards_used** — traceability chain.
- **as_found_in_tolerance** — boolean, whether it was already within spec before adjustment.
- **adjustments_made** — free-text description.
- **notes**.

On save, a **FAIL** result marks the gauge **OUT_OF_SERVICE** — which the measurement picker hides and the server refuses to record against (10a). (A gauge that’s merely *due-soon* or *overdue* but still **IN_SERVICE** stays selectable — that’s the advisory case the gauge-nag tile flags.)

10d — The gauge picker during measurement

§4c walks a measurement substep. On any measurement node, an **Equipment** dropdown sits next to the value field, pre-populated from the measurement’s definition:

- **Default** equipment (tagged “default” in the picker).
- **Backup** equipment (tagged “backup”), if one is configured.

The choice rides along with the reading — the gauge recorded is the actual one used, not the definition’s default. That’s the audit trail hook: three months from now you can trace a measurement back to the specific gauge that produced it, and if that gauge later shows a **FAIL** calibration event, you can walk backwards and find every reading that rode along with it.

Out-of-service filtering. Any option whose equipment is `OUT_OF_SERVICE` at authoring time is hidden from the operator picker; the configured default falls back to nothing (no auto- selection) if that default is itself `OUT_OF_SERVICE`. The server also refuses the write for an `OUT_OF_SERVICE` gauge — so even a stale client that offers a bad option gets rejected at the source, not silently accepted. Due-soon and overdue gauges that are still `IN_SERVICE` remain selectable and rely on the gauge-nag tile for awareness.

10e — Seeded records on the QA walk exhibits

The demo seed creates real calibration records for the equipment used by WO-QA-INSPECT-01 steps (flow bench, torque wrenches, gauges); some dates are intentionally set overdue for demo purposes. That's why the *Overdue* panel and the gauge-nag tile aren't empty on a fresh reseed.

11. The notification bell and inbox

Real day terms: while you're working on one QR, four other things happen around the shop that you should know about. UQMES pushes those to two related but distinct surfaces:

- **Inbox** — things you *have to do*: assigned tasks, approvals waiting on you, dispositions in your queue. Two related surfaces: [/inbox](#) is a generic tabbed personal inbox — CAPA tasks, dispositions, approvals — that anyone with commitments can reach; [/quality/inbox](#) is the QA persona's home page, which is broader than a pure inbox (adds the gauge-nag tile and the My Actions panel alongside the inbox list).
- **Notification feed** (bell → [/notifications](#)) — things you should be *aware of*: events that fired system-wide and your subscriptions routed to you.

The distinction matters. Inbox is your work list; missing something there blocks the shop. The bell is your awareness surface; missing something there just means you didn't see it.

11a — The bell popover

Top-right of the app layout, the **Bell** icon shows an unread count in a small red badge (rendered **99+** if you're really behind).

Clicking opens a popover:

- Header: “*Notifications*” + **Mark all read** button (only when there are unread items).
- Body: last 7 items. Unread rows have a blue dot on the left and render in normal weight; read rows are muted. Each row shows subject + first line of body + relative time (“*just now*”, “*5m ago*”, “*2h ago*”, “*3d ago*”).
- Footer: **View all** navigates to [/notifications](#).

Clicking a row marks it read *and* follows the item's deep link. [ncr.opened](#) sends you to the disposition. [fpi.decided](#) sends you to the FPI banner on the runtime. [capa.assigned](#) sends you to the CAPA detail. If a row has no link, it just marks read.

11b — The full feed at [/notifications](#)

[/notifications](#) — same data as the popover, up to 100 items, with a persistent **Unread only** filter toggle. Header reads “*N unread*” or “*All caught up*” when there's nothing to review. Same click-to-open-and-mark-read behavior as the bell.

Preferences button (top right) navigates to [/profile/notifications](#) — the personal surface where you can:

- Mute or unmute specific events per channel (in-app, email).
- Add personal subscriptions (“*ping me when X happens on records I own*”) — a lighter surface than the admin rule editor at [/settings/notifications](#).
- Watch specific records to get every event that fires on them.

11c — What fires for a QA inspector

On a fresh seed Sarah's bell shows **2 unread**, both `capa.assigned` — “CAPA CAPA-2024-005 assigned to you / Major CAPA, due 2026-08-17” and the same for CAPA-2024-002. Those are the only pre-seeded ones; the rest below fire as you work the walkthrough rather than arriving with the seed.

Events that route to a QA inspector by default (via the tenant's starter notification rules):

- `ncr.opened` (§5c) — a FAIL QR just auto-created a quarantine disposition. Routes to the disposition assignee (a QA Manager or QA Inspector on the tenant).
- `fpi.decided` — an FPI was passed, failed, or waived on a step Sarah covers.
- `capa.assigned` (§9a) — a CAPA task was assigned to you.
- `capa.ready_for_verification` (§9c) — routes to the QA Manager group's inbox after an inspector saves verification data.

Non-QA events (production, escalations, receiving) also flow through the same pipeline; you'll see them if a personal subscription or a watched-records rule routes one to you.

11d — Which surface to check when

- Something specific you're supposed to *finish*? → **Inbox** (or the home page tiles, which are the same data grouped).
- Something changed and you want to know? → **Bell / feed**.
- No time to look at either right now? → glance at the bell's red badge count. If it's non-zero and stays that way, you're ignoring something.

12. Reading the audit trail

Real day terms: an auditor asks you to reconstruct the history of a specific part. Or an operator on the shop floor hands you a physical part and asks “what’s the story on this one?” You need to be able to answer without going into engineering.

12a — A rich in-flight arc (INJ-QA-INSPECT-004)

Open [/details/Parts/...](#) for INJ-QA-INSPECT-004. Reading top-to-bottom:

- **Status** tells you where the part is *right now*.
- **Current Step** tells you where in the process.
- **Latest Inspection** rolls up the most recent QR verdict.
- **Quality Reports** widget lists every inspection (PASS and FAIL) in chronological order. Each row links to the QR detail with the measurement values.
- **Dispositions** widget lists every disposition on the part with state, type, severity, and links to the disposition editor.
- **Activity History** appends every state change with timestamp and actor.

For INJ-QA-INSPECT-004 you can narrate: *fail at Flow Testing on [date] (link to QR); REWORK decision signed by [name] on [date] (link to disposition); reworked (implicit — no operator UI record); returned to Flow Testing visit 2; awaiting Sarah’s re-inspection.*

12b — A closed terminal record (INJ-0042-023)

Open [/details/Parts/...](#) for INJ-0042-023 (existing seed, from Journey 6 of the training script).

Status is **SCRAPPED**. **Quality Reports** has one row — the originating FAIL QR at Final Test with description “Nozzle tip shows visible porosity at seat face; failed hold pressure at 2450 psi (min 2800). Casting defect, not reworkable.” **Dispositions** has the CLOSED SCRAP row ([DISP-QR-0042-023-FT](#), severity CRITICAL, resolution notes citing QP-007). Together those three records ARE the audit trail: cause, decision, terminal state.

If a scrapped part ever shows a status of **SCRAPPED** with **no** linked QR or disposition, that itself is a red flag — how did the part reach a terminal state without a paper trail? Ask before touching.

12c — Two things worth knowing

Multiple dispositions on one QR. QR→disposition is legitimately 1:many — a human can add lines deliberately for different portions of nonconforming material. A FAIL QR also auto-creates a bare disposition; you can spot an auto-created one by its description prefix “Auto-created for failed quality report.”

Sparse Activity History in the seed. Many seeded parts show “No audit history” in the Activity History section because the seed writes state directly rather than going through the operator runtime. The

trail lives on the linked QRs and dispositions in that case. Live parts (those you work during a real day) accumulate a normal history.

13. The manager's side — Maria

\$1–12 walked the floor as Sarah. This one flips to Maria, the QA Manager. She doesn't get a different app — **there is no separate manager dashboard**. She lands on the same QA home page Sarah does (§1). What differs is what routes *to* her: she holds the authority permissions ([approve_disposition](#), [verify_capa](#), CAPA and disposition approval), so the gates Sarah hands up land in Maria's queues. A manager's day is the other side of the inspector's gates.

Log in as maria.qa@demo.ambac.com / [demo123](#).

13a — Your home base: the same QA home, plus /approvals

Two surfaces carry a manager's work.

The QA home (§1), “My quality actions” panel. The same three-tile panel Sarah sees — Approvals · CAPA tasks · My dispositions — but the counts are yours. The **Approvals** tile links to your inbox. Below it, **“Available to claim — your group is eligible, nobody has it”** lists group-routed approvals no one has picked up yet, each with an **Accept** button that moves one into your queue. That's how a request routed to the *QA Manager group* rather than to you by name reaches you.

The Approvals center — /approvals. The dedicated approvals surface, headed **Approvals**. Four stat cards across the top: **Awaiting My Approval** (items requiring your action), **Overdue** (past due date), **My Requests Pending** (things you asked others to approve), and **Recently Approved**. Below them:

- **Awaiting My Approval** — the working list. Each row shows the item, a type badge, who it's from, and a due date (flagged red if overdue). Click a row to open the item itself (a CAPA opens at [/quality/capas/{id}](#)).
- **By Type** — the same pending items bucketed by approval type, so you see “3 CAPA approvals, 1 document” at a glance.
- **My Submitted Requests** — approvals you've asked others for.
- **Quick Links** — Approval History, and Approval Templates (for the manager who configures the workflows themselves).

The four gates below are what those queues are made of.

13b — Authorize a disposition (the other side of §6)

When Sarah picks a disposition type (§6b) she's making an authorized decision she may not hold the authority for, so her editor routes it through a co-sign dialog. If **you** open that disposition — the INJ-QA-INSPECT-003 NCR from §6 — and set its type, it commits directly: you hold [approve_disposition](#), so there's no dialog. Same editor, same **Update Disposition**; the authority is simply already yours. (On a seed you've already walked, §6 authorized that one as a co-sign, so read 13b as the manager's *direct* path — what you'd do on any disposition you authorize yourself.)

Two things the standard forces and the app enforces:

- **USE_AS_IS** and **REPAIR** require a recorded customer/design-approval reference. They accept known-nonconforming product, so the **decide** action refuses them with no reference. **REWORK** and **SCRAP** don't need one.
- The decision is recorded against **you** as the authorizing signature (**decision_authorized_by**) — that's the AS9100/ISO 9001 8.7 signature, not decoration.

13c — Verify a CAPA's effectiveness (the other side of §9c)

Sarah can write a verification *plan*; recording the *outcome* needs **verify_capa**, which is yours — but only once a plan exists. On this seed **CAPA-2024-002** (the §9c exhibit) starts with *no* plan (its Verification tab reads “No verifications have been recorded yet”); §9c has Sarah author one first. Once it's there, open the CAPA → **Verification** tab and its plan row shows **Complete Verification** — click it and record the result. Pick the result — **CONFIRMED** or **NOT_EFFECTIVE** — add notes, and submit.

What your verdict does:

- **CONFIRMED** closes the CAPA.
- **NOT_EFFECTIVE** reopens it to In Progress, flags the RCA for review, and auto-creates a 30-day follow-up task — a correction that didn't stick doesn't quietly close.

You can't verify a CAPA you initiated or were assigned unless self-verification is explicitly enabled with justification — the same segregation of duties that keeps Sarah from signing off her own first piece.

13d — Approve a major CAPA before work starts (the other side of §9c-bis)

A MAJOR or CRITICAL CAPA can't begin work until management approves it. This is the one gate that is purely yours — Sarah only ever sees it read-only (the **Approval** tab's “*Awaiting Approval — work cannot begin until approved*” banner).

On this seed, **CAPA-2024-005** is a MAJOR CAPA waiting on you. It appears in two places at once: your **/approvals** “Awaiting My Approval” list (the CAPA-approval item), and the CAPA's own **Approval** tab. From either:

1. Open it and click **Submit Response**. The **Submit Approval Response** dialog opens.
2. Choose **Approve** (or Reject / Delegate). Reject requires a comment; Delegate hands the request to another user, who becomes the new approver.
3. For Approve, complete the signature block — draw your signature, tick “*I confirm this is my signature and I am authorized to approve this item,*” and enter your password (**demo123**). That is the 21 CFR Part 11 signed approval.
4. Click **Submit Approval**.

The CAPA's approval status flips to APPROVED and work can begin. That request was created for you automatically when the CAPA was raised at MAJOR/CRITICAL severity — you didn't go looking for it; it came to your queue.

13e — Two ways authority reaches you: co-sign vs. your queue

A manager authorizes in two distinct ways. They feel similar — both end in your signature — but they happen at opposite ends of the room, and it's worth keeping them apart.

Co-signature — inline, at the inspector's station, right now. Sarah is standing at a gate she doesn't hold: an FPI pass (§3c), a disposition decision (§6b), a CAPA verification outcome (§9c). Rather than stop and route a request, she calls you over and you authenticate *on her screen* — your email and password in the co-sign fields. You are never logged in as yourself; the act is recorded against you, and Sarah keeps working. Use it when someone is blocked at a gate and you're there to clear it on the spot.

Approval — asynchronous, in your own queue, on your own time. A request is created and *routed* to you (the major-CAPA approval in §13d is the model). It lands in **/approvals**, waits with a due date, and you action it later from your own screen. Nobody is standing at a station waiting on you keystroke by keystroke.

The quick test: **is someone blocked at a gate this second (co-sign), or did a request land in my queue to handle when I get to it (approval)?** Both are your signature; only one interrupts someone else's flow.

14. Authoring a DWI — the process flow

Everything the walk has run through — the receiving DWI (\$2), the FPI gate (\$3), the sampled inspection (\$4), the measurement-driven fail (\$5) — is *authored* somewhere. This section is that somewhere. It's the one part of this guide that isn't a daily inspector task: authoring a process and its work instructions is a **QA-manager / process-author** job (it needs process-edit permission, which a QA Manager like Maria holds). Read it to see where the runtime forms come from and how you'd change one.

14a — Reach the process flow

From the sidebar, **Processes** (</editor/processes>) lists the tenant's processes. This walk seeds two, so you can compare:

- **Injector Reman** — status **Approved**. The live process work orders run against.
- **Injector Reman - Authoring Draft (SHOWCASE)** — status **Draft**. A sandbox copy staged for exactly this section; edit *this* one.

Each row carries three actions: **View** (read-only detail), **Edit Process** (the pencil), and Delete. Click **Edit Process** on the SHOWCASE draft — you land on the **process flow** at </process-flow?id=...>

14b — Read the flow

The flow is a canvas of the process's steps as nodes, wired by routing edges. The injector reman process reads left to right — **Core Receiving** → **Disassembly** → **Component Grading** → **Cleaning** → **Nozzle Inspection** → **Flow Testing** → **Assembly** → **Final Test** → **Packaging** → **Complete** — with a **Rework** station (carrying a visit counter, **? / 3**) that failed parts loop back through, and **Nitride Coating** as the outside-processing branch. The QA steps (Component Grading, Nozzle Inspection, Flow Testing, Final Test) are decision nodes marked **QA Pass/Fail**, showing their **Pass** and **Fail** exits on the node. Each edge is labelled with the outcome that takes it — **Pass**, **Fail**, **Max Exceeded** (the rework limit tripping) — and **Complete** is the terminal **COMPLETED** node. These are the same steps the walk's parts moved through: Nozzle Inspection is \$3/\$4, Flow Testing is \$5.

A **Validation Issues** panel flags authoring errors before you can save or approve — e.g. *"Decision step 'Component Grading' is missing Fail connection"* — so a half-wired branch can't ship.

Above the canvas, a process selector and a part-type selector name what you're editing (here **Injector Reman - Authoring Draft (SHOWCASE) · Common Rail Injector**), and a set of view lenses re-render the same graph as **Process Template** (the authoring view), **Work Order Progress**, **Part Journey**, **Process Evaluation**, and **QA Checkpoints**. Author on Process Template.

14c — Turn on Edit Mode

The page opens read-only ("Process Flow **Viewer**"). Flip the **Edit Mode** switch at the top: the header becomes "Process Flow **Editor**", an **Add Step** button appears, and the nodes and edges become editable

— select and move a node, select an edge and delete it to drop a route, and add steps.

14d — Author a step

Click a step node — say **Nozzle Inspection** (operation 50, “Inspect nozzle spray pattern and wear”) — to open the **Step Details** panel. Top to bottom:

- **Identity** — Name, Operation number, Description, Work center.
- **Decision type** — how the step routes: *Based on QA Pass/Fail*, a measurement, or a manual decision. **Route rejected items to...** picks the destination the **Fail** edge points at — here, **Rework**.
- **Configuration** — five editors, each showing a count of what’s already set: **Measurements**, **Sampling Rules**, **Documents**, **Required Training**, and **Edit substeps**. Measurements and Sampling get their own passes below; Documents attaches drawings/specs, Required Training sets the qualification gate, and **Edit substeps** is where the guided operator steps live — the instruction text and capture nodes (measurement / pass-fail / sign-off) you filled in §2c and §4c. Nozzle Inspection carries 2 measurements, 1 sampling rule, 1 required training, and 2 substeps.
- **Advanced** — the step-behaviour switches whose effects the rest of this guide has been feeling: **Requires QA signoff**, **Sampling required** + min rate %, **Requires first-piece inspection** (the §3 FPI gate), **Max visits (rework limit)** (the §5/§6 rework loop and its “Max Exceeded” edge), Terminal step, Expected duration, Move lot as a unit.

Add Step adds a node; **Delete Step** removes the selected one.

14e — Measurements

Configure measurements opens “Measurements for ” — the readings taken at the step. **Add Measurement** takes:

- **Label** — the field name the operator sees (*Flow Rate*, *Outer Diameter*).
- **Type** — **Numeric** or **Pass/Fail**.
- **Unit** — optional (mL/min, mm, ...).
- **Nominal**, **Upper Tolerance**, **Lower Tolerance** — the target and the spec limits. *These derive the verdict at runtime*: the Flow Rate you entered in §5 failed because it crossed the lower tolerance — there’s no manual Pass/Fail button on a numeric measurement because the spec is the judge.
- **Characteristic #** — the balloon number on the drawing, for traceability.
- **Default / Backup equipment** — the preferred gauge (and a fallback); this is what steers the operator to the right instrument and feeds the point-of-use calibration gate (§10d).
- **Required** — whether the runtime blocks completion without it.

Each measurement authored here becomes a capture field in the operator runtime. Nozzle Inspection carries two; Flow Testing’s single measurement is the flow-rate reading §5 rode.

Why define measurements here and pull them in, rather than typing a spec straight onto the substep. A substep’s Measurement node (§14g) *can* carry its own inline nominal and tolerances, but a defined measurement is what makes the reading useful downstream:

- **SPC** — control charts and Cpk are keyed on the definition, so every reading of that characteristic rolls into one chart; an inline value trends against nothing.
- **Traceability & gauge** — the definition carries the **Characteristic #** (the drawing balloon) and the gauge that feeds the \$10d calibration gate, and it's what a step's Pass/Fail routing can be conditioned on. An ad-hoc field has none of that.

Author the spec once here and reference it; an inline one-off is a dead-end reading.

14f — Sampling

Configure sampling rules opens “Sampling — ”. Not every part gets inspected; this is where you say which ones. Three parts:

- **Sampling method** — **Per-part streaming** (pick individual parts from the flow — every-Nth, %), **Lot acceptance** — **attribute** (a C=0/AQL plan like receiving's, \$2b), or **Variables (measured)**.
- **Primary Sampling Rules** — the normal-operation rate (e.g. *every 4th part*, ~25%). **Escalation Rules** — a tighter rate a run of failures triggers: *escalate after 2 failures → 100%, return after 15 passes → back to ~25% Normal*. That tightening and relaxing is exactly the **Tightened / Reduced** severity badges the inbox shows (\$1), and the reason a part like INJ-QA-INSPECT-002 gets pulled for a sampled look (\$4).
- **Quality gate (automatic escalation)** — a **Metric, Threshold, and Window**, with **Actions when tripped** including **Raise CAPA / SCAR** and **Require approval**. This is the gate behind \$9e: when the fail rate over the window crosses the threshold, the step raises a CAPA on its own, with no human initiator.

14g — Substeps: the DWI itself

The **Substeps** editor — the *Edit substeps* button on the step (\$14d) — is where the operator-facing work instruction actually lives: the guided steps you ran in \$2c and \$4c. It opens as a full page, **Substep Editor**, with the step's substeps listed down the side (Nozzle Inspection's are “0. Visual nozzle inspection” and “1. Measure spray angle”) and **Add substep / Add inspection substep** to add more. Edits stay local until you click **Save draft** (it warns on an unsaved close); **Discard** throws them away.

Each substep has:

- **A title and flags** — *Sign-off* (operator must sign to complete), *Inspection point* (captures here also open a QR + measurement result, firing the out-of-spec → auto-quarantine pipeline you saw in \$5), *Batch (once per lot)*, *Critical* (can never be marked N/A), *Allow N/A*.
- **A body**, written in a rich editor with an insert palette grouped ★ **Frequent · Text & Layout · Capture · Quality · Roles · 3D & Teardown · Templates**. Instruction text sits alongside the capture nodes you drop in — **Measurement, Photo, Sign-off, Scan, Quality status, Callout**, and a **QA inspection bundle** template. Those capture nodes are exactly the fields the operator fills at runtime.

Pulling a measurement in. Drop a **Measurement** node and it starts as a blank inline field — but link it to a measurement you defined at the step (\$14e) instead of typing its spec here. The seeded exhibit does

exactly that: the “Measure spray angle” substep’s node is *Spray Angle · characteristic N-12 · 15 +3 / –3°*, linked to the step’s measurement definition, so its spec, balloon number, and gauge all come from that one definition — and every reading rolls into SPC for that characteristic. Leave it unlinked and it still captures a value, but it’s the dead-end reading §14e warns about. Define the measurement once at the step; pull it into the substep here.

14h — Draft, then approve

You’re editing a **Draft** (the SHOWCASE) on purpose — the live **Injector Reman** is **Approved**, with work orders running against it. The Validation Issues panel (§14b) has to be clear first. Authoring on a draft and approving it into effect is how a process changes without disturbing in-flight work; the two rows on the Processes list are the before/after of that split.

15. Quality Reports

A **Quality Report (QR)** is the record of one inspection outcome — the generated documentation that says *this part, at this step, passed or failed; here is the evidence; here is who took and verified the reading*. You’ve been producing and reading QRs the whole walk (\$5 wrote one on the fail, \$6’s disposition links back to it, \$7 reads the history); this section is the QR itself. QRs are **generated, not hand-authored** — the surface exists to hold and document them, not to build them the way you build a process (\$14).

15a — Where a QR comes from

Three origins, in rough order of how often you’ll meet them:

- **Auto, on a failed inspection** — recording an out-of-spec measurement (\$5) writes a FAIL QR and fires the `ncr.opened` → auto-quarantine pipeline.
- **From an inspection-point substep** — a substep flagged *Inspection point* (\$14g) turns the operator’s captures into a QR as they work.
- **Manually — Quality Reports** (</editor/qualityReports>) → **New Quality Reports**, for a finding that didn’t arrive through a runtime capture (an assessment, an audit observation).

The **Quality Reports** list is the tenant-wide register: *Report # · Status · Part · Step · Detected By · Verified By · Created*, with a status filter, Import / Export, and **View** into each. It sits under </editor/> because it’s the manage-all-records surface — not your daily queue. Your day’s QRs reach you through the part and its disposition (\$5/\$6), not by trawling this list.

15b — Reading a QR

View opens the report, laid out as a formal record:

- **Report** — Report #, **Result** (Pass / Fail), whether it was a First Piece Inspection, and a description.
- **Inspection** — the Part (and its current status), Process, Step, Machine, and the Sampling Method that selected it.
- **Personnel** — **Detected By** (who took the reading) and **Verified By** (the second set of eyes) — the segregation-of-duties record.
- **Findings** — the defect / error types recorded on a failure.
- **Acceptance Sampling** — Sample Size and Accept (Ac) / Reject (Re) when the QR came off a sampling plan (\$2b / \$14f).
- **Attachments, System Information, and Activity History.**

Two actions sit on the report: **Create CAPA** — the \$9d path, escalating a systemic finding straight from the QR — and **Edit Report**. A QR is a controlled record: it’s voided, never hard-deleted (\$12).

15c — Measurements are the evidence on the report

This is the far end of §14e. The measurements you define at a step and pull into its inspection-point sub-steps are the readings that land on the QR: each captured value, set against its nominal and tolerances, is the *evidence* behind the Pass/Fail — not just the verdict. That's the concrete reason to define a measurement once and reference it (§14e) rather than type a loose value: the defined measurement carries its spec, its gauge, and its characteristic number onto the generated report, and lets the reading roll into SPC afterwards. A QR built from ad-hoc inline values documents a number with nothing behind it. (A manually-created QR, or one off a step with no defined measurements, simply has no readings to show — its record is the finding and the sampling result.)

16. Glossary

- **AWAITING_QA** — part state meaning “an operator finished a step but a rule says QA looks at it before it moves on.” Parked; production can’t advance it.
- **CAPA** — Corrective And Preventive Action. Structured investigation and fix for recurring or high-severity defects. Inspectors can initiate CAPAs and work assigned tasks; the final effectiveness verification ([verify_capa](#)) is gated to the QA Manager. See §9.
- **CoC** — Certificate of Conformance. Supplier’s paperwork stating a lot meets the ordered spec. Attached at receiving.
- **DWI** — Digital Work Instruction. On-screen guided capture the operator (and inspector) follows step-by-step.
- **ERP id** — the human-readable identifier (WO-QA-INSPECT-01, INJ-QA-INSPECT-004) shown on labels and travelers.
- **FPI** — First Piece Inspection. First part off a new step, inspected to verify setup before production runs the batch.
- **LSL / USL** — Lower / Upper Spec Limit. Outside these, the part fails.
- **NCR** — Non-Conformance Report. Formal record of a nonconformity. In UQMES an NCR is a [QuarantineDisposition](#).
- **OSP** — Outside Processing. A step run by a subcontractor. Parts are shipped out, worked, returned, and re-inspected.
- **Quarantine** — physical or logical hold on a part while its disposition is decided.
- **QR** — Quality Report. The record of a single inspection outcome (PASS / FAIL / PENDING).
- **Step execution** — one visit to one step by one part. [visit_number > 1](#) means a rework re-visit.
- **Traveler** — the printed WO packet that accompanies the physical parts through the shop. Print it from WO Detail’s Traveler button.
- **WO** — Work Order. A single production run of a specific quantity of one part type for one Order.

Appendix — Sidebar reference

The whole rail a QA user sees, top to bottom (see *Getting around* in the front matter for the two things to know before you use it):

Sidebar section	Entries you'll use	Why you open it
(top, everyone)	Help & Docs (/docs), Tracker (/tracker)	Docs, and the customer-facing tracker map. Not your daily driver.
Personal	Inbox (/inbox)	Your assigned CAPA tasks + approvals, with a live count badge. Two of the home page's "My quality actions" tiles land here.
Production (open)	Work Orders (/production/work-orders), WO Control Center (/workorders), Processes (/editor/processes)	The shop-floor work orders and the process authoring surface.
Supply	Incoming Inspection (/production/incoming), Outside Processing (/production/outside-processing), Materials (/production/material-lots), + supplier & plan surfaces	Receiving (§2) and OSP returns (§8) live here. The receiving-only queue (/production/receiving-inspection, §2a) has no rail entry — reach it from the home Receiving chip.
Remanufacturing	Cores, Components	Reman shops only; skip it if you aren't one.
Quality (open)	Dashboard (/quality), CAPAs (/quality/capas), Quality Reports (/editor/qualityReports), Change Control , Dispositions (/production/dispositions), Training , Calibrations (/quality/calibrations), Heat Map	Your home turf — CAPAs (§9), dispositions (§6), calibration (§10).
Approvals	Overview (/approvals), History	The approvals center (§13a), badge-counted. Mostly a manager's surface.
Tools	Documents , Analytics , AI Chat	Standalone utilities.
Admin	Settings, User Management, Work Centers, Data Management, Audit Log	Tenant admins only — a QA inspector won't see this section at all.